RUN: OOP Unit 2, Polymorphism and Overloading

Nov 5, 2026 · @shahan

All of Unit 2, the most reliably examined unit in OOP. Allow about 2 hours. **Every program here was compiled and run with g++.** Almost every paper hands you a `main()` and says "make this run", so this is really one skill practised ten ways.

How this run works

Read the idea, write the PYQ yourself, then compare. Lines like `c2 = --c1;` tell you exactly which operators to write: read the `main()` first and list them.

Videos: in the CppNuts playlist, watch Operator Overloading, Unary Operator Overloading, Array Subscript Operator Overloading, Stream Insertion and Extraction Overloading, and Overload TypeCast. Slides are in `OOP/Notes/Unit 2 slides.pdf`.

Stop 1: Polymorphism

> **PYQ, Sessional-1 2026 Q1(b), this year's paper; Sessional-1 2024 Q3, 5 marks.** Define polymorphism. Differentiate between compile-time and run-time polymorphism, with code.

Answer: Polymorphism means one name with many forms.

Basis	Compile-time	Run-time
Also called	Static or early binding	Dynamic or late binding
Achieved by	Function and operator overloading	Virtual functions
Decided by	Parameter types	Actual object type
Speed	Faster	Slight cost, through a vtable

```
int add(int a, int b) { return a + b; }           // compile-time
double add(double a, double b) { return a + b; }

class Shape { public: virtual void draw() { cout << "Shape"; } };
class Circle : public Shape { public: void draw() { cout << "Circle"; } };

Shape* s = new Circle;
s->draw();     // prints Circle: run-time
```

Compile-time polymorphism is overloading, resolved by parameter types. Run-time polymorphism is virtual functions, resolved by the object's actual type.

Stop 2: Operator overloading rules

Operators are functions with better PR. `a + b` becomes `a.operator+(b)`.

Kind	Member form	Friend form
Binary	One argument	Two arguments
Unary	No arguments	One argument

Can't be overloaded: `::`, `.`, `.*`, `?:`, `sizeof`. **Must be members:** `=`, `[]`, `()`, `->`. **Must be friend or non-member:** `<<` and `>>`, because the left operand is `cout`, and that's not your class. Precedence and the number of operands can't change.

> **PYQ, Sessional-1 2026 Q3, this year's paper, 4 marks.** Implement class Product with private price and a constructor, so that this main runs: `Product p1(1000), p2(500), p4(1000); Product p3 = p1 + p2;` then `p3.display();`, `if (p1 > p2)` and `if (p1 == p4)`.

Answer:

```
#include <iostream>
using namespace std;

class Product {
    int price;
public:
    Product(int p = 0) : price(p) {}
    Product operator+(const Product& o) const { return Product(price + o.price); }
    bool operator>(const Product& o) const { return price > o.price; }
    bool operator==(const Product& o) const { return price == o.price; }
    void display() const { cout << price << endl; }
};

int main() {
    Product p1(1000), p2(500), p4(1000);
    Product p3 = p1 + p2;
    cout << "Sum of prices: ";
    p3.display();
    if (p1 > p2)
        cout << "Product 1 is costlier" << endl;
    if (p1 == p4)
        cout << "Product 1 and Product 4 have the same price" << endl;
    return 0;
}
```

Output: `Sum of prices: 1500`, `Product 1 is costlier`, `Product 1 and Product 4 have the same price`.

Three operators are needed: `+` returning a new Product, and `>` and `==` returning bool.

> **PYQ, Sessional-2 2025 Q1, 4 marks.** Implement class Counter with private count, constructors and get_count(), so that the given main runs: `Counter c1(3), c2(3);`, then `c2 = --c1;`, then `c2 = c1--;`, printing both after each step.

Answer: The main needs prefix and postfix `--`. The postfix version takes a dummy `int`, and returns the old value.

```
#include <iostream>
using namespace std;

class Counter {
    int count;
public:
    Counter() : count(0) {}
    Counter(int c) : count(c) {}
    int get_count() const { return count; }
    Counter operator--() {               // prefix: change first, return the new value
        --count;
        return *this;
    }
    Counter operator--(int) {            // postfix: the int is a dummy that marks it
        Counter old = *this;
        count--;
        return old;
    }
};

int main() {
    Counter c1(3), c2(3);
    cout << "\n c1 = " << c1.get_count();
    cout << "\n c2 = " << c2.get_count();
    c2 = --c1;
    cout << "\n c1 = " << c1.get_count();
    cout << "\n c2 = " << c2.get_count();
    c2 = c1--;
    cout << "\n c1 = " << c1.get_count();
    cout << "\n c2 = " << c2.get_count() << endl;
    return 0;
}
```

Output: c1 = 3, c2 = 3, then c1 = 2, c2 = 2, then c1 = 1, c2 = 2. In the last step c2 gets the old value, which is the entire point of the question.

Final values: c1 = 1, c2 = 2. Prefix `--` returns the new value; postfix `--(int)` returns the old one.

> **PYQ, Sessional-1 2024 Q2, 5 marks.** Implement class Time with hours, minutes, seconds so that `Time T1(2, 45, 50); Time T2(1, 30, 40); Time T3 = T1 + T2;` and `cout << "T1 = " << T1` run.

Answer:

```
#include <iostream>
using namespace std;

class Time {
    int h, m, s;
public:
    Time(int a = 0, int b = 0, int c = 0) : h(a), m(b), s(c) {}
    Time operator+(const Time& t) const {
        int total = (h + t.h) * 3600 + (m + t.m) * 60 + s + t.s;
        return Time(total / 3600, (total % 3600) / 60, total % 60);
    }
    friend ostream& operator<<(ostream& out, const Time& t) {
        return out << t.h << ":" << t.m << ":" << t.s;
    }
};

int main() {
    Time T1(2, 45, 50);
    Time T2(1, 30, 40);
    Time T3 = T1 + T2;
    cout << "T1 = " << T1 << endl;
    cout << "T2 = " << T2 << endl;
    cout << "T3 = " << T3 << endl;
    return 0;
}
```

Converting to total seconds handles every carry at once, so there's nothing to forget.

$$T_1 = 2(3600) + 45(60) + 50 = 9950 \text{ s}$$
$$T_2 = 1(3600) + 30(60) + 40 = 5440 \text{ s}$$
$$T_3 = 15390 \text{ s} = 4 \text{ h } 16 \text{ min } 30 \text{ s}$$

T3 = 4:16:30.

> **PYQ, Endsem 2025 Q2(a), 4.5 marks.** Implement class Date with day, month, year so that `Date D1(28, 2, 2024); Date D2(31, 12, 2023);`, `cout << D1`, and `++D1; ++D2;` work.

Answer: The dates were chosen as traps: 2024 is a leap year, and 31 December rolls over the year. A lazy `d++` gets 0 marks' worth of correct output.

```
#include <iostream>
using namespace std;

class Date {
    int d, m, y;
    bool leap() const { return (y % 4 == 0 && y % 100 != 0) || y % 400 == 0; }
    int daysInMonth() const {
        int days[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
        return (m == 2 && leap()) ? 29 : days[m - 1];
    }
public:
    Date(int a, int b, int c) : d(a), m(b), y(c) {}
    Date& operator++() {
        if (++d > daysInMonth()) {
            d = 1;
            if (++m > 12) { m = 1; y++; }
        }
        return *this;
    }
    friend ostream& operator<<(ostream& out, const Date& t) {
        return out << t.d << "/" << t.m << "/" << t.y;
    }
};

int main() {
    Date D1(28, 2, 2024);
    Date D2(31, 12, 2023);
    cout << "Original D1: " << D1 << endl;
    cout << "Original D2: " << D2 << endl;
    ++D1;
    ++D2;
    cout << "After Increment D1: " << D1 << endl;
    cout << "After Increment D2: " << D2 << endl;
    return 0;
}
```

After the increments: D1 = 29/2/2024 (2024 is a leap year) and D2 = 1/1/2024 (the year rolls over).

> **PYQ, Endsem 2025 Q2(c), 4.5 marks.** Implement all functions for class Matrix with `int mat[10][10]`, rows, cols, default and parameterized constructors, input(), display(), `operator*`, `operator~` (transpose), and `operator==`.

Answer:

```
#include <iostream>
using namespace std;

class Matrix {
    int mat[10][10];
    int rows, cols;
public:
    Matrix() : rows(0), cols(0) {}
    Matrix(int r, int c) : rows(r), cols(c) {
        for (int i = 0; i < r; i++)
            for (int j = 0; j < c; j++) mat[i][j] = 0;
    }
    void input() {
        for (int i = 0; i < rows; i++)
            for (int j = 0; j < cols; j++) cin >> mat[i][j];
    }
    void display() {
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) cout << mat[i][j] << " ";
            cout << "\n";
        }
    }
    Matrix operator*(Matrix& b) {
        if (cols != b.rows) {
            cout << "Cannot multiply\n";
            return Matrix();
        }
        Matrix r(rows, b.cols);
        for (int i = 0; i < rows; i++)
            for (int j = 0; j < b.cols; j++)
                for (int k = 0; k < cols; k++)
                    r.mat[i][j] += mat[i][k] * b.mat[k][j];
        return r;
    }
    Matrix operator~() {
        Matrix t(cols, rows);
        for (int i = 0; i < rows; i++)
            for (int j = 0; j < cols; j++) t.mat[j][i] = mat[i][j];
        return t;
    }
    bool operator==(Matrix& b) {
        if (rows != b.rows || cols != b.cols) return false;
        for (int i = 0; i < rows; i++)
            for (int j = 0; j < cols; j++)
                if (mat[i][j] != b.mat[i][j]) return false;
        return true;
```

```
    }
};
```

Test it with two 2 × 2 matrices:

$$A \times B = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

$$\sim A = A^T = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}$$

Zeroing `r` in the constructor is what makes `+=` safe.

`*` checks that A's cols equal B's rows, `~` swaps rows and cols, and `==` compares the sizes first, then every element.

> **PYQ, Sessional-2 2022 Q1, 4 marks.** Design class Complex, overloading `+=`, `-=`, `*=`, `/=` and `<<`, using the given formulas.

Answer:

```
#include <iostream>
using namespace std;

class Complex {
    double x, y;
public:
    Complex(double a = 0, double b = 0) : x(a), y(b) {}
    Complex& operator+=(const Complex& c) { x += c.x; y += c.y; return *this; }
    Complex& operator-=(const Complex& c) { x -= c.x; y -= c.y; return *this; }
    Complex& operator*=(const Complex& c) {
        double r = x * c.x - y * c.y;
        y = x * c.y + y * c.x;
        x = r;
        return *this;
    }
    Complex& operator/=(const Complex& c) {
        double d = c.x * c.x + c.y * c.y;
        double r = (x * c.x + y * c.y) / d;
        y = (y * c.x - x * c.y) / d;
        x = r;
        return *this;
    }
    friend ostream& operator<<(ostream& out, const Complex& c) {
        return out << c.x << " + i(" << c.y << ")";
    }
};
```

Note the temporary `r`. Updating `x` first would corrupt the value `y` still needs, a bug that's easy to write and hard to spot.

$$(x_1 + iy_1)(x_2 + iy_2) = (x_1x_2 - y_1y_2) + i(x_1y_2 + x_2y_1)$$

$$\frac{x_1 + iy_1}{x_2 + iy_2} = \frac{(x_1x_2 + y_1y_2) + i(x_2y_1 - x_1y_2)}{x_2^2 + y_2^2}$$

With a = 3 + 4i and b = 1 + 2i, the sequence prints 4 + 6i, 3 + 4i, −5 + 10i, 3 + 4i.

Stop 3: The subscript operator

`operator[]` must be a member. Return a reference if the element should work on both sides of `=`.

> **PYQ, Sessional-2 2025 Q2, 4 marks.** Create a template class SafeArray for any data type. Overload `[]` for bounds checking, with an error message for an invalid index. In main, create int and float SafeArrays, store values and display them.

Answer:

```
#include <iostream>
using namespace std;

template <class T>
class SafeArray {
    T* a;
    int size;
public:
    SafeArray(int n) : size(n) { a = new T[n](); }
    ~SafeArray() { delete[] a; }
    T& operator[](int i) {              // returns a reference, so it works on the left of =
        if (i < 0 || i >= size) {
            cout << "Error: invalid index " << i << "\n";
            return a[0];                // keeps the program safe instead of crashing
        }
        return a[i];
    }
};

int main() {
    SafeArray<int> x(3);
    SafeArray<float> y(2);
    x[0] = 10; x[1] = 20; x[2] = 30;
    y[0] = 1.5; y[1] = 2.5;
    for (int i = 0; i < 3; i++) cout << x[i] << " ";
    cout << "\n";
    for (int i = 0; i < 2; i++) cout << y[i] << " ";
    cout << "\n";
    cout << x[5] << "\n";
}
```

Output: `10 20 30`, `1.5 2.5`, then the error message, followed by 10. A reference has to refer to something, so the bad case returns `a[0]`. Say so in your answer.

`template <class T>`, with `T& operator[](int i)` that checks 0 ≤ i \< size before returning the element.

> **PYQ, Sessional-2 2022 Q4, 3 marks.** Overload `[]` for class AccountBook holding 5 account holders. An account number should print the holder's name, and a name should print the account number.

Answer: Two overloads, differing by parameter type.

```
#include <iostream>
#include <string>
using namespace std;

class AccountBook {
    string names[5];
    int nums[5];
public:
    AccountBook() {
        string n[] = {"Asha", "Ravi", "Sana", "Imran", "Zoya"};
        for (int i = 0; i < 5; i++) { names[i] = n[i]; nums[i] = 101 + i; }
    }
    string operator[](int acc) {               // account number gives the name
        for (int i = 0; i < 5; i++)
            if (nums[i] == acc) return names[i];
        return "Not found";
    }
    int operator[](string name) {              // name gives the account number
        for (int i = 0; i < 5; i++)
            if (names[i] == name) return nums[i];
        return -1;
    }
};

int main() {
    AccountBook b;
    cout << b[103] << "\n";
    cout << b[string("Zoya")] << "\n";
}
```

Output: `Sana`, `105`. The paper asks for input, so replace the constructor's fixed names with a `cin` loop in the exam.

Two `operator[]` overloads: one takes an int and returns the name, the other takes a string and returns the number.

> **PYQ, Sessional-1 2023 Q3(b), 3 marks.** Define class Feature with a 256-dimensional int vector, initialised to random values. Overload `[]` so it works on the LHS and RHS. Make an array of 10,000 Feature objects and compute the centroid.

Answer:

```
#include <iostream>
#include <cstdlib>
using namespace std;

class Feature {
    int v[256];
public:
    Feature() { for (int i = 0; i < 256; i++) v[i] = rand() % 100; }
    int& operator[](int i) { return v[i]; }   // a reference works on both sides of =
};

int main() {
    const int N = 10000;
    Feature* data = new Feature[N];
    data[0][0] = 7;                           // LHS use
    double centroid[256] = {0};
    for (int i = 0; i < N; i++)
        for (int j = 0; j < 256; j++) centroid[j] += data[i][j];   // RHS use
    for (int j = 0; j < 256; j++) centroid[j] /= N;
    delete[] data;
}
```

The centroid is the mean along each dimension:

$$c_j = \frac{1}{10000} \sum_{k=1}^{10000} f_k[j], \qquad j = 0, 1, \ldots, 255$$

Using `new` is deliberate: 10,000 objects of 256 ints is about 10 MB, too big for the stack.

Returning `int&` from `[]` is what lets it work on both the LHS and the RHS.

> **PYQ, Sessional-1 2023 Q2(b), 2.5 marks.** Define class StrReplicate with a string member. Write the overloaded prefix `++` that appends another copy of the string to itself.

Answer:

```
class StrReplicate {
    string s;
public:
    StrReplicate(string t) : s(t) {}
    StrReplicate& operator++() {
        s += s;
        return *this;
    }
    void show() const { cout << s << "\n"; }
};
```

`++r` on "ab" gives "abab". Returning a reference lets `++(++r)` work, and applying that gives 8 copies of "ab".

`StrReplicate& operator++() { s += s; return *this; }`**, so "ab" becomes "abab".**

Stop 4: Friend overloading

> **PYQ, Sessional-2 2022 Q3(d), 1 mark.** Explain overloading through a friend function.

Answer: It's needed when the left operand isn't your class. `m + 5` works as a member, but `5 + m` can't, because `5` is an `int`.

```
class Money {
    int rs;
public:
    Money(int r) : rs(r) {}
    friend Money operator+(int a, const Money& m);   // the left operand is not a Money
    void show() const { cout << rs << "\n"; }
};
Money operator+(int a, const Money& m) { return Money(a + m.rs); }
```

`(5 + Money(100)).show()` prints `105`.

A friend operator takes both operands as parameters, so the left operand doesn't have to be your class.

> **PYQ, Endsem 2025 Q2(b), 4.5 marks.** Explain when a friend function should take arguments by reference, and when by value. Design classes Account (balance) and Loan (amount). Create a friend function deductLoan(...) that deducts the loan from the balance.

Answer: Use **reference** when the function must change the original, or when the object is large. Use **const reference** when it's only read, to avoid the copy. Use **value** for small data when the function needs its own copy to work with. A friend gets no `this`, so objects arrive only through parameters, which is why this choice matters.

```
#include <iostream>
using namespace std;

class Loan;

class Account {
    double balance;
public:
    Account(double b) : balance(b) {}
    double getBalance() const { return balance; }
    friend void deductLoan(Account&, const Loan&);
};

class Loan {
    double amount;
public:
    Loan(double a) : amount(a) {}
    friend void deductLoan(Account&, const Loan&);
};

// Account by reference, because the change must reach the original.
// Loan by const reference, because it is only read and copying is pointless.
void deductLoan(Account& a, const Loan& l) {
    a.balance -= l.amount;
}

int main() {
    Account a(10000);
    Loan l(2500);
    deductLoan(a, l);
    cout << a.getBalance() << "\n";
}
```

Output: `7500`. Pass Account by value and the balance doesn't change, which is a very bad property in a bank.

`friend void deductLoan(Account& a, const Loan& l)`**: Account by reference because it changes, Loan by const reference because it's only read.**

Stop 5: Conversions

From → To	How
Basic → class	Single-argument constructor
Class → basic	Conversion function: `operator float()`
Class A → class B	Constructor in B taking an A, or `operator B()` in A

> **PYQ, Sessional-1 2023 Q2(a) and Q3(a), 5 marks.** Define TempC and TempF, each with a float attribute. Write a conversion from TempF to TempC, using C = (F − 32) × 5/9. Overload `<<` for TempC to print "X degrees Celsius". What does this print, and why? `TempF summer(118); cout << summer;`

Answer:

```
#include <iostream>
using namespace std;

class TempF {
    float f;
public:
    TempF(float x = 32) : f(x) {}
    float get() const { return f; }
};

class TempC {
    float c;
public:
    TempC(float x = 0) : c(x) {}
    TempC(const TempF& t) : c((t.get() - 32) * 5 / 9) {}   // conversion constructor
    friend ostream& operator<<(ostream& out, const TempC& t);
};

// Defined outside the class, so it can be found for a TempF argument too
ostream& operator<<(ostream& out, const TempC& t) {
    return out << t.c << " degrees Celsius";
}

int main() {
    TempF summer(118);
    cout << summer;
}
```

$$C = \frac{(118 - 32) \times 5}{9} = \frac{430}{9} = 47.7778$$

TempF has no `<<` of its own, so the compiler converts it to TempC using the conversion constructor, and then calls TempC's `<<`.

Output: `47.7778 degrees Celsius`.

A trap I hit for real: if you define that friend `<<` *inside* TempC, this code doesn't compile. A friend defined inside a class is only found when an argument is already that class. Testing caught it, and the examiner would have too.

That's Unit 2. On to Unit 3, where classes start having children and all the associated problems.